

流程控制（上）：条件判断

本章学习目标

- 掌握 if-else 语句的各种形式
 - 理解条件判断的执行流程
 - 熟练使用 case 语句进行多分支选择
 - 学会处理复杂的条件嵌套
 - 能够编写交互式菜单脚本
 - 掌握条件判断的最佳实践
-

5.1 if 语句基础

5.1.1 if 语句的基本语法

if 语句是最基本的条件判断结构，其基本语法如下：

```
if condition; then
    commands
fi
```

或者写成多行形式（更常见）：

```
if condition (true 或 false)
then
    commands
fi
```

关键点：

- `condition` 是条件表达式，通常是 `[ ]`、`[[ ]]` 或 `(( ))` 表达式
- `then` 关键字必须存在，如果和 `if` 同一行，需要用 `;` 分隔
- `fi` 是 `if` 的反写，表示条件语句结束

- 条件为真（退出状态为 0）时执行 `commands`

【实操练习 5-1】最简单的 if 语句

```
#!/bin/bash
# 文件名: simple_if.sh

age=20

# 形式1: condition 和 then 同一行
if [ $age -ge 18 ]; then
    echo "你已成年"
fi

# 形式2: condition 和 then 分行
if [ $age -ge 18 ]
then
    echo "你可以投票了"
fi

# 使用 [[ ]]
if [[ $age -ge 18 && $age -le 60 ]]; then
    echo "你在工作年龄范围内"
fi

# 使用 (( ))
if (( age >= 18 )); then
    echo "使用双括号也可以判断"
fi
```

5.1.2 条件判断的本质

在 Shell 中，条件判断的本质是检查命令的退出状态码：

- 退出状态码为 0 表示成功/真
- 退出状态码为非 0 表示失败/假

```
#!/bin/bash
```

```
# 任何命令都可以作为条件
if grep -q "root" /etc/passwd; then
    echo "找到 root 用户"
fi

# 直接使用命令
if ping -c 1 -W 1 127.0.0.1 > /dev/null 2>&1; then
    echo "网络正常"
fi

# 检查上一条命令的退出状态
ls /nonexistent 2>/dev/null
if [ $? -ne 0 ]; then
    echo "目录不存在"
fi
```

5.2 if-else 语句

5.2.1 if-else 基本形式

当需要在条件不满足时执行其他操作，使用 if-else：

```
if condition; then
    commands1
else
    commands2
fi
```

【实操练习 5-2】if-else 使用示例

```
#!/bin/bash
# 文件名: if_else.sh

read -p "请输入你的年龄： " age

# 验证输入是否为数字
if [[ ! $age =~ ^[0-9]+$ ]]; then
    echo "错误：请输入有效的数字！ "
```

```

        exit 1
    fi

# if-else 判断
if [ $age -ge 18 ]; then
    echo "你已成年，可以观看此内容"
else
    echo "抱歉，你未满18岁，无法观看"
fi

```

5.2.2 if-elif-else 多分支

当有多个条件需要判断时，使用 elif：

```

if condition1; then
    commands1
elif condition2; then
    commands2
elif condition3; then
    commands3
else
    commands4
fi

```

【实操练习 5-3】成绩等级判断

```

#!/bin/bash
# 文件名: grade.sh
# 功能: 根据分数判断成绩等级

read -p "请输入考试分数(0-100): " score

# 输入验证
if [[ ! $score =~ ^[0-9]+$ ]]; then
    echo "错误: 请输入数字! "
    exit 1
fi

if [ $score -lt 0 ] || [ $score -gt 100 ]; then

```

```
    echo "错误：分数必须在0-100之间！"
    exit 1
fi

# 成绩等级判断
if [ $score -ge 90 ]; then
    grade="A (优秀)"
elif [ $score -ge 80 ]; then
    grade="B (良好)"
elif [ $score -ge 70 ]; then
    grade="C (中等)"
elif [ $score -ge 60 ]; then
    grade="D (及格)"
else
    grade="F (不及格)"
fi

echo "你的成绩等级是：$grade"
```

5.3 条件判断的高级技巧

5.3.1 使用 && 和 || 简化条件判断

对于简单的条件判断，可以使用短路运算符代替 if 语句：

```
# 传统写法
if [ -f /etc/passwd ]; then
    echo "文件存在"
fi

# 简化写法
[ -f /etc/passwd ] && echo "文件存在"

# 带 else 的简化写法
[ -f /etc/passwd ] && echo "存在" || echo "不存在"
```

⚠️ **注意：** `&&` 和 `||` 组合使用时要小心，如果 `&&` 后面的命令失败，`||` 后面的命令也会执行。

```
# 危险示例
[ -f /etc/passwd ] && command_may_fail || echo "失败"
# 如果 command_may_fail 返回非零，"失败"也会被打印

# 安全写法
if [ -f /etc/passwd ]; then
    command_may_fail
else
    echo "失败"
fi
```

5.3.2 字符串判空的最佳实践

```
str=""

# 方法1：使用 -z（推荐）
if [ -z "$str" ]; then
    echo "字符串为空"
fi

# 方法2：使用 -n 的否定
if [ ! -n "$str" ]; then
    echo "字符串为空"
fi

# 方法3：与空字符串比较
if [ "$str" = "" ]; then
    echo "字符串为空"
fi

# 方法4：使用 [[ ]]（更安全，变量无需引号）
if [[ -z $str ]]; then
    echo "字符串为空"
fi
```

5.4.3 数字判断的最佳实践

```
num="42"

# 方法1: 使用正则验证是否为数字
if [[ $num =~ ^[0-9]+$ ]]; then
    echo "$num 是正整数"
fi

# 方法2: 验证带符号整数
if [[ $num =~ ^-?[0-9]+$ ]]; then
    echo "$num 是整数"
fi

# 方法3: 验证小数
if [[ $num =~ ^-?[0-9]*\.[0-9]+$ ]]; then
    echo "$num 是数字"
fi

# 在算术运算前验证
read -p "请输入数字: " input
if [[ ! $input =~ ^[0-9]+$ ]]; then
    echo "错误: 请输入有效数字"
    exit 1
fi
result=$((input * 2))
echo "结果: $result"
```

5.4.4 文件判断的常见模式

```
#!/bin/bash
# 文件名: file_check_patterns.sh

file="/path/to/file"

# 模式1: 检查文件存在后再操作
if [ -f "$file" ]; then
    cat "$file"
else
```

```
    echo "文件不存在: $file"
    exit 1
fi

# 模式2: 使用短路运算符
[ -f "$file" ] || { echo "文件不存在"; exit 1; }
cat "$file"

# 模式3: 检查目录并创建
dir="/path/to/dir"
if [ ! -d "$dir" ]; then
    mkdir -p "$dir"
    echo "目录已创建: $dir"
fi

# 模式4: 检查权限后操作
if [ -w "$file" ]; then
    echo "追加内容" >> "$file"
else
    echo "没有写入权限: $file"
fi

# 模式5: 检查多个条件
config="/etc/myapp/config.conf"
if [ -f "$config" ] && [ -r "$config" ]; then
    source "$config"
else
    echo "配置文件不可用, 使用默认配置"
    # 使用默认值
fi
```

5.5 case 语句

当需要对一个变量的多个可能值进行匹配时, case 语句比 if-elif-else 更加清晰。

5.5.1 case 语句的基本语法

```
case $variable in
    pattern1)
```



```

        commands1
        ;;
    pattern2)
        commands2
        ;;
    pattern3|pattern4)
        commands3
        ;;
    *)
        default_commands
        ;;
esac

```

语法要点：

- `case` 和 `esac`（case 的反写）配对使用
- 每个模式以 `)` 结尾
- 每个分支以 `;;` 结束
- `|` 可以匹配多个模式
- `*` 匹配任何值，通常作为默认分支
- 模式支持通配符：`*`、`?`、`[...]`

【实操练习 5-4】基础 case 语句

```

#!/bin/bash
# 文件名: case_basic.sh

read -p "请输入一个数字(1-3): " num

case $num in
    1)
        echo "你选择了 1 - 第一项"
        ;;
    2)
        echo "你选择了 2 - 第二项"
        ;;
    3)
        echo "你选择了 3 - 第三项"
        ;;

```

```
*)
    echo "无效的选择！请输入1-3之间的数字"
    ;;
esac
```

5.5.2 case 语句的模式匹配

```
#!/bin/bash
# 文件名: case_patterns.sh

read -p "请输入文件名: " filename

case $filename in
    # 匹配特定扩展名
    *.txt)
        echo "这是一个文本文件"
        ;;
    *.sh)
        echo "这是一个Shell脚本"
        ;;
    *.tar.gz|*.tgz)
        echo "这是一个gzip压缩的tar包"
        ;;
    *.zip|*.rar|*.7z)
        echo "这是一个压缩文件"
        ;;
    # 匹配以特定字符开头
    log*)
        echo "这可能是一个日志文件"
        ;;
    # 匹配单个字符
    ?)
        echo "文件名只有一个字符"
        ;;
    # 匹配字符范围
    [a-z]*)
        echo "文件名以小写字母开头"
        ;;
    [0-9]*)
        echo "文件名以数字开头"
```

```

        ;;
    *)
        echo "未知文件类型"
        ;;
esac

```

5.5.3 使用 case 处理命令行选项

```

#!/bin/bash
# 文件名: case_options.sh
# 功能: 处理命令行选项

show_help() {
    echo "用法: $0 [选项]"
    echo ""
    echo "选项:"
    echo "  -h, --help      显示帮助信息"
    echo "  -v, --version   显示版本信息"
    echo "  -s, --start     启动服务"
    echo "  -p, --stop      停止服务"
    echo "  -r, --restart   重启服务"
}

# 检查是否有参数
if [ $# -eq 0 ]; then
    show_help
    exit 1
fi

# 处理第一个参数
case $1 in
    -h|--help)
        show_help
        ;;
    -v|--version)
        echo "版本: 1.0.0"
        ;;
    -s|--start)
        echo "正在启动服务..."
        # 启动逻辑

```

```
;;
-p|--stop)
    echo "正在停止服务..."
    # 停止逻辑
    ;;
-r|--restart)
    echo "正在重启服务..."
    # 重启逻辑
    ;;
*)
    echo "错误: 未知选项 '$1'"
    echo ""
    show_help
    exit 1
    ;;
esac
```

5.5.4 case 语句 vs if-elif-else

场景	推荐使用	原因
单变量多值匹配	case	代码更清晰
范围判断（数值比较）	if-elif	case 不支持数值范围
需要模式匹配（通配符）	case	原生支持
复杂条件组合	if-elif	更灵活
菜单选择	case	代码简洁

5.7 条件判断最佳实践

5.7.1 代码风格建议

```
# 1. 保持一致的缩进（推荐4个空格或1个Tab）
if [ condition ]; then
    commands
```

```

fi

# 2. 复杂条件拆分为多行
if [ $condition1 ] && \
    [ $condition2 ] && \
    [ $condition3 ]; then
    commands
fi

# 3. 使用有意义的变量名
if [ $user_age -ge 18 ]; then      # 好
if [ $a -ge 18 ]; then            # 不好

# 4. 添加注释说明复杂逻辑
# 检查用户是否有权限：必须是管理员或属于 wheel 组
if [ "$user" = "root" ] || groups "$user" | grep -q wheel; then
    echo "有权限"
fi

```

5.7.2 常见错误及避免方法

错误类型	错误示例	正确示例
缺少空格	<code>[\$a -eq \$b]</code>	<code>[\$a -eq \$b]</code>
变量未加引号	<code>[\$str = hello]</code>	<code>["\$str" = "hello"]</code>
混用比较符	<code>[\$a > \$b]</code>	<code>[\$a -gt \$b]</code> 或 <code>[[\$a > \$b]]</code>
缺少 then	<code>if [cond] commands</code> <code>fi</code>	<code>if [cond]; then commands fi</code>
算术比较用 =	<code>[\$a = 5]</code> (想比较数字)	<code>[\$a -eq 5]</code>

5.7.3 性能考虑

```
# 1. 简单条件用 [ ], 复杂条件用 [[ ]]
[ -f /etc/passwd ]          # 简单文件测试
[[ $str == *pattern* ]]    # 需要模式匹配

# 2. 避免不必要的子进程
# 不好 (创建子进程)
if [ $(echo "$str" | grep -c pattern) -gt 0 ]; then

# 好 (Bash 内置)
if [[ $str == *pattern* ]]; then

# 3. 合理使用短路求值
# 替代简单的 if 语句
[ -d /tmp/mydir ] || mkdir -p /tmp/mydir
```

5.8 本章小结

本章我们学习了 Shell 脚本中的条件判断：

if 语句：

1. 基本形式：`if condition; then commands fi`
2. if-else：两个分支选择
3. if-elif-else：多分支选择
4. 可以嵌套使用，但要控制层级

条件测试：

1. 条件的本质是命令的退出状态码（0为真）
2. 使用 `[ ]`、`[[ ]]` 或 `(( ))` 进行测试
3. `[[ ]]` 更强大，支持模式和正则匹配

case 语句：

1. 适合单变量多值匹配
2. 支持通配符模式匹配
3. 每个分支以 `;;` 结束

4. `*`) 作为默认分支

最佳实践：

1. 变量要用双引号括起来
2. 避免过深的嵌套
3. 使用有意义的变量名
4. 适当添加注释